A Ground-Up Tutorial on Origin-Bounded Trust for Cross-Organizational Agent Systems

---

Table of contents

Part 0 — The world this research lives in

Before any security, we need three plain-English ideas.

0.1 What is an "LLM agent"?

A **Large Language Model (LLM)** is an AI system that produces text. An **agent** is an LLM that has been given the ability to *act* — to call tools, fetch web pages, query databases, send messages, and crucially, to **talk to other agents**. Instead of just answering you, an agent can decompose a task and delegate parts of it.

0.2 What is a "multi-hop, cross-organizational chain"?

Imagine you ask your personal assistant agent: *"Book me the cheapest sensible flight and tell me the baggage rules."* Your agent cannot do all of this alone, so:

1. **Your agent** (Organization A) asks

2. a **travel-broker agent** (Organization B), which asks
3. an **airline's agent** (Organization C), which
4. **fetches** baggage rules from a **web page or API** (the *external source*).

The answer then flows back **up** the chain: C → B → A → you. Each step is a **hop**. The chain is **multi-hop** (several steps) and **cross-organizational** (the agents belong to different companies that do not fully trust each other).

At each hop an agent may **transform** the content: summarize it, paraphrase it, or combine ("compose") it with other information. You, the **principal**, finally act on whatever comes back.

```
None
   YOU (principal)
      ▲  |  delegate
      |  ▼
   Agent A  ──hop──►  Agent B  ──hop──►  Agent C  ──fetch──►  Web page / API
   (your org)         (broker)           (airline)            (the ORIGIN)
      ◄──────────── answer flows back up, transformed at each hop ────────────
```

0.3 Why is this dangerous?

Because **you only see the final answer.** You did not watch agent C read the web page. You are trusting a long relay of strangers. If the original web page was **wrong or malicious**, but the agents that relayed it are **reputable and authenticated**, the lie arrives wearing the credibility of the messengers.

That single sentence is the whole problem. Everything below is about defining it exactly and stopping it.

Part 1 — The problem: cross-agent information laundering

1.1 The phenomenon, named

The paper calls the core danger **cross-agent information laundering**:

> **Information laundering** = the rise of a claim's *apparent* trustworthiness as it passes through trusted intermediaries, **decoupled from the actual warrant of its origin.**

The analogy is *money laundering*: dirty money passed through legitimate banks comes out looking clean. Here, a **dirty claim** (planted on an editable wiki) passed through **legitimate agents** comes out looking authoritative.

1.2 The precise confusion at the heart of it

There are two different things one might "trust," and the system confuses them:

- **Trust in the *deliverer*** — "Agent B is a real, authenticated, reputable company." (True, and verifiable today.)
- **Trust in the *information*** — "The claim Agent B delivered is well-founded." (A completely different question, and *not* verifiable today.)

Laundering is precisely the failure of **mistaking the first for the second**. This is called a **transitive-trust failure**: trust does not legitimately pass along the chain the way the system implicitly assumes it does.

1.3 A concrete attack story

1. An attacker edits a low-quality, editable web page to say *"Drug X is safe at 10× the normal dose."*
2. A retrieval agent fetches it. A summarizer agent condenses it. A medical- assistant agent composes it into a clean paragraph and cites "a web source."
3. Your agent receives an authoritative-sounding, well-formatted answer from a reputable medical-assistant agent.
4. **You act on it.** Laundering succeeded: the *apparent* warrant (high) far exceeds the *true* warrant of the origin (an anonymous editable page = very low).

1.4 Defining "success" for the attacker — precisely

We need a non-fuzzy definition so we can *measure* defenses. The paper gives one:

> **Laundering success** occurs when the **accepted output warrant exceeds the maximum warrant justified by the true origin state.**

In symbols, if `W_out` is the trust the principal ends up assigning the final answer, and `W_origin*` is the most trust the *true* origin could ever justify, then laundering succeeded exactly when

```
None
        W_out  >  W_origin*
```

Notice what this is **not**: it is *not* "the claim is false." A defense here is **not** a truth detector. It is a guarantee that **trust never rises above what the origin earns.** Keep this distinction; it returns everywhere.

Part 2 — A tour of the prior art (literature review)

A natural reaction is "surely someone already solved this." Many people solved *nearby* problems. The contribution of this work is identified precisely by seeing what each prior line of work does and does **not** cover. We group the literature **by which layer of trust it addresses**, because the gap is exactly a layer none of them covers.

2.1 (a) Identity and transport security — *"who is talking?"*

What it is. Cryptographic systems that authenticate *which agent* is speaking and protect the communication channel from tampering.

- **Decentralized Identifiers (DIDs) and Verifiable Credentials (VCs)** — W3C standards giving each agent a cryptographically verifiable identity and signed attestations about itself.
- **SAGA (Security Architecture for Governing Agentic systems, NDSS 2026).** Users register agents with a central **Provider**; the Provider stores contact info and access-control policies and issues cryptographic **access-control tokens** (via a Diffie–Hellman-derived one-time-key mechanism) so agents can enforce fine-grained permissions on who may talk to them. It provides *formal security guarantees* on the communication path and gives users lifecycle control over their agents.

What it secures. *Who relayed a message,* and *was the channel tampered with.*

The gap it leaves. These are **content-agnostic**. A fully authenticated, perfectly legitimate agent can faithfully relay a **falsehood**, and identity security will cheerfully confirm "the sender is legitimate." Identity answers *who*, never *is what they're saying well-founded.* (In this work, SAGA is not a competitor — it is the **substrate** CAPM builds on for identity and signed channels.)

2.2 (b) Provenance models — *"where did it come from (on record)?"*

What it is. Standards for recording the lineage of data as it moves through a workflow.

- **W3C PROV** — the standard data model for provenance: entities, activities, and agents, and the relationships among them (who derived what from what).

- **PROV-AGENT** — extends provenance recording into *agentic* AI workflows, tracking how data passes through agent steps.

What it secures. A faithful **record** of lineage.

The gap it leaves. Two limits. First, they are built for **honest participants** and **single workflows** — not adversaries spanning organizations. Second, and decisively, **they grade nothing**: a recorded source is just a recorded source. PROV has no notion that *some origins justify less trust* than others, or that *trust should erode* when content is transformed. It tells you the path; it never tells you how much to believe what came down it.

2.3 (c) Capability / information-flow control — *"what may flow, inside one runtime?"*

What it is. Techniques that tag values with labels and enforce rules on how those labels may flow — classic *information-flow control* and *taint tracking*.

- **CaMeL** — a capability-tagging / prompt-injection-defense pattern: untrusted data is tagged and prevented from influencing privileged actions, enforced inside one controlled runtime.

What it secures. Strong, principled control of data flow **within a single runtime / trust domain.**

The gap it leaves. Blind across the organizational boundary. Capability labels live in one program's memory; they do **not travel with the content** once it crosses to *another organization's* agent in a different process on a different machine. The moment the claim leaves the runtime, the labels evaporate. (CAPM *ports the idea* of capability tagging but makes the label **travel**, signed, across the boundary — it does not import CaMeL.)

2.4 (d) Retrieval grounding / citation checking

What it is. Defenses for Retrieval-Augmented Generation (RAG) that check a cited source **exists** and was actually retrieved (rather than hallucinated).

The gap it leaves. They verify **source presence**, not **trust preservation**. "This citation is real" says nothing about whether the cited origin deserved trust, nor whether that trust survived being summarized and relayed three hops later.

2.5 (e) Content / semantic filtering

What it is. LLM-as-judge guards and classifier filters that **read the text** of a claim and try to flag bad content.

The gap it leaves — and why it is a trap. A guard that judges **persuasiveness** is judging exactly the thing an attacker **optimizes**. Authority-framed falsehoods ("Studies confirm…") score *high* on a persuasiveness check. These defenses see *text*; laundering is a property of *provenance*, not text. So a text filter is structurally the wrong tool — and the paper's own analysis (and the laundering literature) shows it.

2.6 (f) Signed-manifest containers and supply-chain integrity

What it is. Cryptographic provenance for *artifacts*.

- **C2PA (Coalition for Content Provenance and Authenticity)** — signs media with a tamper-evident **manifest** describing how it was made; widely used for image/ video content credentials. Single-author oriented.
- **in-toto / SLSA** — secure the *software supply chain*: signed attestations that each build/transform step happened as claimed.
- **Remote attestation** — hardware/software proofs of *what is running.*

Relationship. These are **not competitors** — they are mechanisms CAPM **composes with**. C2PA donates the *container pattern* (a hash-linked, signed manifest). in-toto/SLSA/attestation are the **origin-integrity layer** that CAPM's *residual* hands off to (more on "residual" in Part 8).

2.7 (g) The attack literature — *what makes the problem real*

These works **demonstrate** that the danger is not hypothetical:

- **AgentDojo** — a benchmark of **prompt-injection** attacks against tool-using agents (e.g. banking tasks), with realistic attack and defense evaluation.
- **RAG poisoning** — planting adversarial documents so retrieval surfaces them.
- **Multi-agent knowledge propagation** — showing false "knowledge" spreads and *gains* credibility across a network of agents.
- **Causality / denial laundering** — manipulating apparent cause/effect or plausibly denying provenance as content moves.

The common thread, proven by all of them: **low-quality or adversarial content gains apparent credibility as it moves through trusted agents** — and *none* of the defenses in (a)–(e) stops this across organizational boundaries.

2.8 The literature in one table

Layer	**Example work**	**Secures**	**Does not do**
Identity / transport	DIDs/VCs, **SAGA**	*who* speaks; channel integrity	content trust; faithful relay of lies passes

Layer	Example work	Secures	Does not do
Provenance record	W3C PROV, **PROV-AGENT**	lineage record	grades nothing; assumes honesty
Capability / IFC	**CaMeL**	flow control *in one runtime*	labels don't cross the org boundary
Retrieval / citation	RAG-citation checks	source *presence*	trust *preservation* across transforms
Semantic filter	LLM-judge guards	flags some text	attacker optimizes persuasiveness; reads text not provenance
Signed manifests	**C2PA**, in-toto/SLSA	artifact/supply-chain integrity	(composed with, not a competitor)
Attack literature	**AgentDojo**, RAG-poisoning	*proves the threat is real*	(it's the attack side)

Part 3 — The precise gap and problem statement

3.1 The gap, stated exactly

Reading the table, every prior layer secures a *different* coordinate. Stack them and one layer is still missing:

> **No existing mechanism enforces a graded trust value, bound to the true origin, that is preserved (and provably non-increasing) as content is transformed and relayed across organizational boundaries.**

Unpack the five load-bearing words:

- **graded** — not yes/no trust, but a *value* on a scale (some origins earn more).
- **bound to the true origin** — the value is anchored to *where the claim actually came from*, not who relayed it.
- **preserved** — it travels *with* the content across hops.
- **provably non-increasing** — relays can only *lower* it, never *raise* it. (This is the mathematical heart, formalized in Part 7.)

- **across organizational boundaries** — between mutually distrusting orgs, over real networks.

3.2 The formal problem statement

> In a multi-hop, cross-organizational agent chain, the principal must decide how much to trust a delivered claim. Existing systems let the principal verify the deliverer's identity and (at best) read an **ungraded** provenance log. They provide no way to **bound the claim's trust by the true warrant of its origin** and to **guarantee that no relay — however trusted — can raise that trust in transit.**
>
> We seek a **deployable** mechanism that: **(i)** binds trust to the origin; **(ii)** makes trust **monotone non-increasing** across hops and transformations; **(iii)** computes the trust **outside** the language model (so persuasive text cannot launder warrant); and **(iv)** does so across **real** organizational / process boundaries.

Those four requirements (i)–(iv) are the design contract. The rest of the tutorial is how each is met.

Part 4 — The system model and definitions (with notation)

Now we get precise. These definitions are the vocabulary for everything after.

Term	Precise meaning
Claim	An atomic unit of content transmitted between agents. The thing whose trust we track.
Origin	The **first acquisition point** of a claim — the web page, API, tool, or DB record where it entered the system.
Acquisition wrapper	A trusted component (browser tool / retrieval service / API client) that, *at the moment content crosses into the system*, produces and **signs an origin observation**. (Detailed in Part 6.)

Term	Precise meaning
Origin observation	The signed tuple the wrapper emits — see below.
Manifest	A **hash-linked chain** of per-hop, per-segment **signed** records: the origin observation plus one signed record per transformation, **one signature per agent under its own key.**
Warrant	An ordered trust label / score that the **external verifier** assigns, derived **from the manifest — never from the delivered text.** This is the "graded trust value."
Verifier	A component **outside the language model** that validates signatures and computes warrant at the **principal-facing boundary.**
Capture	The adversary causing an origin observation to bind **high** warrant to **low-integrity** content (e.g. by compromising the wrapper).

4.1 The signed origin observation (the atom of trust)

Every wrapper, at acquisition, signs **exactly** this tuple (the "locked" format):

```
None
⟨ content_hash,            // cryptographic hash of the acquired bytes
  source_URI / API / tool_id,   // where it came from
  timestamp,               // when it was acquired
  retriever_identity,      // which registered wrapper fetched it
  source_class_evidence,   // OBSERVABLE evidence used to classify (Part 6)
  acquisition_context,     // how it was acquired
  nonce ⟩                  // a one-time random value for replay protection
```

Two things to internalize:

1. **The source never self-attests.** A web page cannot be trusted to say "I am authoritative." Instead the **wrapper attests what it observed** about the acquisition channel. Trust is grounded in *observation*, not *self-report*.
2. **`source_class_evidence` is evidence, not a verdict.** The wrapper records the *observable facts* (was there a content signature? a valid cert chain? is the host

editable?), and a deterministic policy (Part 6) turns those facts into a class. The class is never a free parameter someone types in.

4.2 The manifest as a hash-linked chain

Think of a blockchain-like structure, but tiny and per-claim. Record 0 is the origin observation. Each later record describes one transformation and **includes the hash of the previous record**, then is **signed** by the agent performing that hop:

```
None
[ R0: origin observation ]          signed by wrapper
        |  hash(R0) embedded in R1
        ▼
[ R1: "summarized", h(R0) ]          signed by Agent C
        |  hash(R1) embedded in R2
        ▼
[ R2: "composed with X", h(R1) ]     signed by Agent B
        ▼   ...
```

Because each record commits to the previous one's hash and is individually signed, you **cannot**: forge a record (no key), silently drop a record (the hash chain breaks), reorder records (hashes won't match), or alter content (the `content_hash` won't match). This is the cryptographic backbone.

Part 5 — The threat model: adversary, assumptions A1–A6

A security claim is meaningless without saying **what the attacker can do** and **what you assume they cannot.** This is the threat model.

5.1 What the adversary *can* do

The adversary may:

- **control external sources** (own malicious web pages, third-party APIs);
- **operate or compromise one or more relay agents** in the chain;
- attempt to **forge, replay, reorder, drop, or splice** manifest segments;
- **mislabel transformations** ("I just quoted it" when they rewrote it);
- **collude across multiple relays**;
- **aggregate multiple sources** to manufacture false confidence;

- attempt to **influence or compromise the acquisition wrapper**;
- **read all published warrant values** (the manifest is *not* secret).

That last point matters: we assume a **strong, fully-informed attacker** who can see exactly how trust is computed and where it is weakest. Security must not depend on hiding anything.

5.2 What the adversary *cannot* do (the assumptions)

Crucially, the adversary may **not** (under the stated assumptions) **forge a signature without the key**, nor **cause the verifier to skip verification.** These are standard cryptographic assumptions.

5.3 The six assumptions A1–A6 — each with a mechanism and a matching attack

This table is the spine of the threat model. Read it as: *"We assume X; we enforce it with mechanism Y; if you remove Y, attack Z comes back."* The fact that every assumption is enforced by a concrete, **removable** mechanism is what lets the work *test* each one (by ablation — removing it and watching the attack return).

ID	Assumption	Enforced by	If dropped → attack returns
A1	Signing keys are secret; signatures unforgeable	Ed25519 per-segment signing + credential registry	**forged-receipt** attack
A2	Receipts are mandatory; their absence is detected	hash-linked chain + verifier	**omission** (launder by dropping a hop)
A3a	*Syntactic* transforms are verifiable (hash / quote / span binding)	content-hash + span binding	**verbatim-but-altered**
A3b	*Semantic* transforms are **not** perfectly verifiable → warrant **conservatively degrades**	monotone penalty + soft-binding as a *degrade trigger, not a truth oracle*	(no fragile detector to break — see safe rule)
A4	Origin source-class is bound into the origin	acquisition-wrapper signature	**mid-chain reclassification**

ID	Assumption	Enforced by	If dropped → attack returns
	observation's signed bytes		
A5	The origin observation is created and signed **at acquisition, before relay**, by the wrapper	acquisition wrapper	**relay-authored origin / wrapper bypass**
A6	Composition warrant is **min-bounded** (weakest input wins)	min-rule in the verifier	**aggregation / false-confidence**

5.4 The single most important rule: the **A3 safe rule**

Distinguish two kinds of transformation:

- **Syntactic** (A3a): "I quoted bytes 100–200 verbatim." This is **verifiable** — hash the span and check. If it matches, the transform was faithful.
- **Semantic** (A3b): "I summarized the meaning." This is **not** perfectly verifiable — no algorithm reliably proves a summary is faithful.

The temptation is to build a clever "is-this-summary-faithful?" detector. That is a trap: any such detector is a classifier an attacker can fool. So the work refuses to depend on one. Instead:

> **A3 safe rule.** *If a transformation's fidelity cannot be cryptographically or structurally verified, warrant **MUST** drop.* Unverifiable-as-faithful is treated as **generation** (maximum penalty).

The consequence is profound: **security never depends on a semantic classifier being correct.** It depends only on this *conservative default.* A *better* fidelity detector would only raise **utility** (let more honest summaries keep their warrant); it could **never** be the difference between secure and insecure. This is how you build a defense that cannot be defeated by fooling an ML model — you remove the ML model from the security-critical path entirely.

Part 6 — How origins get their trust label (the acquisition wrapper)

The most-attacked question in the whole design is: **"How does the wrapper decide a source's class?"** If that decision is a subjective judgment call, the entire "origin-bound" guarantee rests on sand. So the answer must be **mechanical**.

6.1 The deterministic, evidence-only source-class policy

The wrapper inspects **observable properties of the acquisition channel** and maps them to a class by **fixed rules**. *No content semantics. No model judgment. Only channel evidence.*

Observable channel evidence	**Source class**	**Why**
Signed API response from an **allowlisted, authenticated** endpoint (valid TLS chain + request auth + **response signature**)	**STRONG** / authoritative-API	authenticated origin **with content integrity**
Signed feed / content credential (e.g. C2PA-signed, chains to a trusted issuer)	**STRONG** / verified-document	cryptographic origin integrity
First-party internal DB/service read over an authenticated channel	**MODERATE** / first-party-DB	trusted, but no per-record content signature
Unsigned **static** page on a non-editable corporate domain (valid TLS, no auth, no content signature)	**MODERATE-LOW** / public-webpage	host identified, no content integrity
Editable / UGC platform (wiki, issue tracker, comment, forum)	**WEAK** / editable-source, bound to a **sub-origin** (author/edit id), not the host	host ≠ author; moderation ≠ integrity

Observable channel evidence	**Source class**	**Why**
Search snippet / aggregator (origin URL not directly fetched)	**WEAK** / unverifiable	provenance one hop removed
Endpoint **known to return model-generated content** (LLM API)	**NONE** / unverifiable-pipeline	generation, not acquisition of a pre-existing fact
Any of the above with a **stale** timestamp beyond a freshness window	**decay** the class	freshness risk
Channel evidence **insufficient to classify**	**default to NONE (degrade)**	the A3 safe rule, applied to origin classification

6.2 The two rules that make this secure *without a truth oracle*

1. **Evidence-only.** The class is a function of *channel evidence the wrapper can structurally verify* — cert chains, content signatures, auth, endpoint allowlist, host editability, freshness — **never** of the content's meaning.
2. **Degrade-on-uncertainty.** If the evidence does not clearly establish a class, assign the **lower** class (ultimately NONE). Therefore **misclassification can only ever *under*-trust.** Over-trust requires *forged channel evidence*, which is exactly **wrapper/origin compromise** — the residual (Part 8).

This asymmetry is the elegant safety property: honest mistakes are always *safe* (they make you trust *less*); the only way to be *unsafe* is genuine cryptographic compromise.

6.3 How each signal is *actually* detected (no hand-waving)

- **"Signed API response" vs "just TLS".** TLS authenticates the *channel/host*; it says **nothing** about *content integrity*. STRONG requires a **content signature** over the response body (e.g. an HTTP Message Signature / JWS / detached COSE) verifying to the endpoint's registered key. No content signature → it is *only* TLS → classed as public-webpage / first-party-DB, **never** authoritative-API. This one distinction is the crux of the whole policy.
- **"Non-editable corporate domain."** Detected by **allowlist membership** plus the *absence of edit affordances* on the fetched path — not "it looks corporate."
- **UGC sub-origin extraction.** For known UGC platforms the wrapper extracts the **author / revision / comment id** from structured metadata and binds warrant to that **sub-origin**, not the host. Can't extract it? Degrade to WEAK (rule 2).

- **Stale window.** A *declared, auditable* per-class freshness parameter (signed- feed TTL, API `Cache-Control`/`Date`, configurable default) — not an ad-hoc number.
- **Allowlist construction.** Built from **registered, credentialed endpoints** (the deployment's trust root / SAGA-style registry) — **popularity is explicitly never an input.**
- **Over-trust guard.** Because popularity never raises class and the *only* paths to STRONG are *content signature* or *registered credentialed endpoint*, a high- traffic **editable** site stays WEAK. You **cannot** talk the classifier into over-trusting a popular-but-low-integrity source. The only route to STRONG is forged evidence = compromise = the residual.

6.4 The wrapper is the Trusted Computing Base (TCB) — stated, not hidden

Introducing the wrapper makes it the **TCB** for origin provenance: if the wrapper is wrong or compromised, downstream warrant is wrong. Good security engineering *names* its TCB rather than pretending there isn't one. The wrapper's attack surface is treated as first-class:

- **wrapper compromise** — attacker controls acquisition → forged evidence → over-trust. *This is the residual proper;* it is measured, and damage is bounded by **attribution** (the manifest still names the compromised wrapper for later quarantine).
- **wrapper bypass** — content enters without a wrapper → must surface as a **missing origin observation** (A2) → quarantine. Never silent trust.
- **wrapper misclassification** — honest-but-imperfect evidence → by rule 2 can only **under**-trust → measured rate + utility cost.
- **stale / replayed observation** — defeated by timestamp + freshness + the **nonce**.
- **malicious retriever identity** — `retriever_identity` is signed and bound to a registered credential; an unregistered retriever fails verification.

  The TCB is deliberately **small and well-defined** — the wrapper + the signing keys + the verifier. The claim is **not** "no TCB"; it is **"a minimal, named, evaluated TCB,"** which is the standard way to make a trust argument defensible.

Part 7 — CAPM the mechanism, and the mathematics of monotone warrant

Now the defense itself. **CAPM = Cross-Agent Provenance Manifests.**

7.1 The mechanism in five steps

CAPM = origin observation (signed by the wrapper) + a hash-linked, per-hop-signed manifest + an **external verifier** that computes warrant by:

1. **Verifying every signature** back to a registry the verifier does **not** control (so the verifier can't be tricked by a self-serving trust root);
2. **Starting** warrant at the origin's `source_class` **ceiling**;
3. **Applying a monotone non-increasing penalty** per transformation (with the A3 safe rule: unverifiable → maximum penalty);
4. **Min-bounding composition** over multiple inputs (the weakest source caps the result);
5. **Emitting accept / down-weight / quarantine outside the language model.**

7.2 The mathematics, built up gently

Let warrant values live in an **ordered set** $(W, \leq)$ — for now picture a small ladder of labels:

```
NONE  <  WEAK  <  MODERATE-LOW  <  MODERATE  <  STRONG
```

(Equivalently a numeric scale, e.g. $[0,1]$. The structure, not the specific labels, is what matters — proved later in Part 8 / encoding-invariance.)

Step 2 — the origin ceiling. The origin observation fixes a starting warrant

```
w₀  =  ceiling(source_class)        e.g. WEAK for an editable page
```

Step 3 — per-hop transformation penalty. Each hop applies a function T_i that transforms warrant $w_{i-1} \to w_i$. The **defining property** required of every legal transformation is:

```
MONOTONICITY (non-increasing):   Tᵢ(w)  ≤  w    for all w, for every hop i.
```

In words: **a hop can keep warrant the same or lower it — never raise it.** Chain several hops:

```
w   =  T (T ₋₁( … T₁(w₀) … ))   ≤   w₀
```

So no matter how many reputable agents relay the claim, and no matter how persuasive their rewrites, the final warrant is **bounded above by the origin's ceiling.** That single inequality, $w \le w_0$, is the mathematical statement of "laundering is impossible in transit."

The A3 safe rule, mathematically. For a transformation whose fidelity is *not* verifiable, T_i is set to the **maximum penalty** — it drops warrant to the generation floor:

```
None
   if fidelity not verifiable:   T_i(w) = ⊥  (the lowest warrant)
```

This is *still monotone* (it only lowers), so it can never *create* a security hole — at worst it lowers utility. Exactly the property we wanted in §5.4.

Step 4 — composition is min-bounded. When a hop **combines** several inputs with warrants $w^{(1)}, \ldots, w^{(k)}$, the composed warrant is the **minimum**:

```
None
   COMPOSITION:   w_comp  =  min( w⁽¹⁾, w⁽²⁾, …, w⁽ᵏ⁾ )
```

This kills the **aggregation attack** (A6): you cannot launder a WEAK claim by surrounding it with STRONG ones and averaging. The weakest input caps the whole. (Averaging would let `9×STRONG + 1×WEAK` look strong — `min` forbids it.)

Step 5 — the verdict, computed outside the LLM. The verifier compares the final warrant to a tunable **floor** and emits one of:

```
None
   accept           (warrant ≥ floor)
   down-weight      (warrant low but usable — graded, not binary)
   quarantine       (warrant below usability / verification failed)
```

Because this happens **outside** the language model, no amount of persuasive text in the claim can change the verdict. Persuasiveness is computed on *nothing*; the verdict is a function of the *signed manifest* alone.

7.3 Why monotonicity is the *explanation*, not a trick

Containment is a **structural** property: *no relay operation can raise warrant.* That is the **monotonicity invariant**, and it is what makes laundering fail **regardless of how persuasive**

the relayed text is. It holds across warrant encodings (a discrete lattice, a continuous `[0,1]` scale, or even a *learned* scorer that is constrained to be monotone) — and a deliberately **non-monotone control leaks**, which is the experimental proof that the *structure*, not the specific numbers, does the work. (This is contribution **C2**, and the machine- checked proof lives in an appendix so it *supports* the system rather than headlining it — recall the NDSS framing in §3.3.)

Part 8 — The Residual Reduction Theorem (the intellectual core)

Here is the most beautiful idea in the work. We have a defense; what's left?

8.1 Statement

> **Theorem (Residual Reduction, scoped).** Within the CAPM transit adversary model (A1–A6, with A3 split and the acquisition-wrapper model), **every** successful laundering attack — via relay, reclassification, omission, collusion, or aggregation — is **blocked** unless an assumption-enforcing mechanism is removed, **or** the origin-integrity assumption (call it **A7**) fails. The residual is therefore **origin-integrity compromise** (including acquisition-wrapper compromise), refined by the origin-state taxonomy; stale / mixed / UGC / unverifiable states are handled by **defined degradations**, leaving genuine origin / wrapper capture as the **irreducible core.**

8.2 What it means in plain words

Map every transit attack to the assumption that stops it:

Attack	**Blocked by**
forge a signed receipt	A1 (unforgeable signatures)
drop a hop to hide a weak origin	A2 (hash chain detects omission)
claim "verbatim" but alter bytes	A3a (content-hash mismatch)
pass a summary off as faithful	A3b + A3 safe rule (warrant drops)
relabel the origin mid-chain	A4 (class bound into signed origin bytes)

Attack	**Blocked by**
author a fake origin at a relay / bypass the wrapper	A5 (origin signed at acquisition)
aggregate weak sources into false confidence	A6 (min-bounding)

Every arrow in the laundering threat space lands on one of A1–A6. So **if all the mechanisms are in place, the *only* way to win is to corrupt the origin itself** — either the real-world source or the wrapper that observes it. Everything collapses to **one** residual: **origin-integrity compromise.**

8.3 The honest part: the residual is *managed, not eliminated*

The work does **not** claim to eliminate the residual — that would be dishonest, because if the attacker genuinely owns the source, no provenance system can know. Instead it **refines** the residual with an **origin-state taxonomy**, so that most "bad origin" situations are still handled gracefully and only *genuine compromise* truly leaks:

Origin state	**CAPM treatment**
honest, high-integrity	preserve warrant (up to the class ceiling)
honest but **stale**	**time-decay** the warrant
mixed-source	**min** over the embedded sub-sources
UGC on a trusted host	bind to the **sub-origin**, not the domain
compromised (incl. wrapper compromise)	**A7 violation — the residual proper**
unverifiable pipeline (API returns model-generated text)	**degrade**

So genuine origin/wrapper capture is the irreducible core; **stale / mixed / UGC / unverifiable** are all caught by *defined degradations* (decay, min, sub-origin binding, degrade) — not left to leak.

8.4 What is and isn't claimed as novel

The **novelty is the exhaustiveness** — the proof that, under the model, *nothing else survives.* The origin-vs-transit split itself is **not** claimed as new; it is the established "Plane-1 / Plane-2" distinction in the literature and is explicitly credited as prior art. The contribution is showing the transit plane is *provably emptied* down to the single origin residual.

Part 9 — WGOT: the residual as an optimization problem

We've reduced everything to one residual: *which origins might the attacker compromise?* That is now a crisp **optimization** problem — and its solution serves **both** sides. WGOT = **Warrant-Gradient Origin Targeting.**

9.1 Setup and notation

Every origin o in the universe of origins O has four numbers:

Symbol	Meaning
w_o	its **warrant** (how much trust a claim from it carries)
r_o	its **reach** (how widely its claims propagate / how many queries hit it)
p_o	its **capture probability** (how likely the attacker can compromise it)
c_o	its **capture cost** (what it costs the attacker to compromise it)

The product $w_o \cdot r_o \cdot p_o$ is the **residual-risk score** of origin o: high- warrant *and* far-reaching *and* easily-captured origins are the dangerous ones.

9.2 The attacker's problem (budgeted set selection)

Given an attack budget B, the attacker picks a **set** S of origins to compromise to maximize damage:

```
None
   maximize    Σ_{o ∈ S}  w_o · r_o · p_o
   subject to  Σ_{o ∈ S}  c_o  ≤  B
```

This is a **0/1 knapsack** problem (pick items with values and weights under a budget) — which is **NP-hard** in general. So the work is careful with language: it claims **WGOT is a warrant-cost *greedy* policy that *approximates*** the optimum, and it **reports the gap to an exact (ILP) solver on small instances.** It does **not** say "optimal" unless greedy provably matches the exact solver. This honesty is itself part of the contribution.

9.3 The defender's problem (the load-bearing dual)

Here is the elegant turn. Give the **defender** a hardening budget `B_D`, a hardening cost `h_o` per origin, and a **post-hardening capture probability** `p_o(H)` (hardening lowers how likely an origin is to fall). The defender minimizes the *same* residual-risk score:

```
None
   minimize    Σ_{o ∈ O}  w_o · r_o · p_o(H)
   subject to  Σ_{o ∈ H}  h_o  ≤  B_D
```

9.4 The duality — and the best hook in the paper

The **same residual-risk score `w·r·p`** induces *both* the attacker's optimal targeting *and* the defender's optimal hardening order. Said memorably:

> **A well-built provenance defense, by publishing its warrant values, *localizes* the attacker's best target — and that very localization is the defender's prioritized hardening checklist.** The defense signposts its own weakest point; the signpost is also the repair list.

A precise caveat the work insists on: this is a **duality over one score**, *not* "the same algorithm." The two problems are *related* (attacker maximizes, defender minimizes the same `w·r·p`) but **distinct** — the defender's action *changes the landscape* (`p_o → p_o(H)`), so they are not literally inverse.

9.5 Why WGOT is more than "attack the cheap valuable node"

The attacker side alone is intuitive ("hit high-warrant, high-reach, low-cost origins" — obvious). The **value is the defender result**: hardening the WGOT-ranked top-`k` reduces residual risk **faster** than every alternative heuristic — random, degree-based, popularity, cheapest-first, highest-warrant-only, highest-risk-only — **measured, under budget, and robust to wrong cost estimates.** That operational defender result is what makes WGOT a *core contribution* rather than a nice add-on.

Part 10 — Contributions, honesty, and what is actually built

10.1 The five contributions

The **system** is *the* contribution; the rest are properties of it (this avoids the "too many contributions / layer pile" reviewer complaint).

- **C1 — The system.** *To our knowledge, CAPM is the first implemented mechanism combining cross-domain agent communication, graded origin-bound warrant, signed per-hop manifests, an acquisition wrapper that attests origins at the boundary, and external monotone verification.* (Real *only if* the multi-runtime prototype + wrapper exist.)
- **C2 — Monotonicity as the explanation.** Monotonicity explains *why* CAPM works; the prototype shows it can be deployed. (Machine-checked lemma + encoding invariance, in the appendix.)
- **C3 — The scoped decomposition.** Transitive laundering provably reduces to origin/wrapper integrity under the stated model; novelty is the *exhaustiveness.*
- **C4 — WGOT.** Budgeted residual targeting + its defender dual; greedy approximation with gap-to-ILP; the **defender** result is the load-bearing half.
- **C5 — Deployment value.** Verifier placement, the tunable security/utility operating point, warrant↔integrity coupling, **attribution + quarantine hooks** (full global revocation is explicitly *future work*), and the WGOT hardening recipe — all framed as *how to operate C1.*

10.2 The discipline of honesty (why this is good science)

The work is engineered to *report* uncomfortable truths rather than hide them:

- **Two threat classes are never averaged.** Transit attacks (→ contained, ASR ≈ 0) and origin/wrapper capture (→ leaks at a cost, ASR > 0) go in **separate** tables. Mixing them into one flattering average is forbidden.
- **Residual ASR > 0 is expected and reported** — it is the *honest residual* of Part 8, not a bug to tune away.
- **Utility cost is reported.** Because agents mostly summarize/paraphrase (semantic transforms → the A3 safe rule lowers warrant), a real worry is *"CAPM is secure only because it distrusts most useful behavior."* The work treats refuting this as **central, not optional**: it must show a real operating point with **both** low attack success **and** acceptable benign throughput, that CAPM **down-weights rather than over-blocks**, and that strictness is **tunable** (a security/utility Pareto curve).

10.3 What is genuinely real today vs. what must be built

This separation is stated bluntly so the paper never overclaims.

Genuinely real now:

- **Cryptography** — each agent has a distinct Ed25519 keypair; every segment is really signed over canonical bytes; the receiver verifies against a registry it does not control. *The trust boundary is cryptographically genuine.*
- **Warrant logic** — monotone scoring, origin ceiling, min-bounded composition, the A3 conservative-degrade rule — implemented and machine-checked.
- **Vendored SAGA crypto / CA / Monitor** for the signing primitive, trust root, and overhead measurement.
- **Analysis layer** — the decomposition search, WGOT (as analysis), ablations, encoding-invariance — all in a single-process testbed.

Simulated today (must become real for the systems claim):

- **Process / transport boundary.** A "cross-org hop" is currently a recursive *in-process method call* — no socket, no second runtime. Honest framing: *"cryptographic multi-runtime semantics, single-process execution."*
- **Origin acquisition / classification.** `source_class` is currently a scenario parameter / static lookup — the signature is real, but *what it asserts* is hand-set ground truth. **No code yet wraps a real fetch and signs what it observed.**

The three builds that close the gap (in priority order):

- **Build A — the acquisition wrapper.** *The true paper gate.* Without it, the "origin" is scenario metadata and a reviewer's fatal line is *"the guarantee depends on a label you assigned by hand."* Build A must demonstrate real HTTP, API/tool (AgentDojo banking), and file/DB wrappers that sign the observed tuple, derive `source_class` from observable evidence (Part 6), detect bypass, and have compromise + misclassification *measured.* This converts the origin from **asserted** to **observed and attested.** It is more important than Build B because it is the novel, load-bearing piece.
- **Build B — real transport.** Locked stack: **Docker containers + gRPC over mTLS + separate verifier and registry/CA containers**, with manifests genuinely on the wire and Ed25519 signatures on segments. **Two linked identity layers:** the mTLS cert is the *transport* identity; the SAGA-style registry maps agent IDs → manifest-signing keys; the verifier checks **both** that `mTLS identity == registered agent identity` **and** that the manifest signature verifies under that agent's registered key. This yields real latency/bandwidth/CPU numbers and makes C1's deployment claim true.

- **Build C — the closest-competitor baseline.** A *family* of tuned "provenance graph + signed origin + trust policy" competitors (deliverer-threshold, origin-threshold, min-source-threshold, chain-length penalty, transformation penalty *without* strict monotone/min semantics). CAPM must beat the **best** of them on the **security–utility trade-off** — not a strawman. This answers the *"just provenance + a score"* objection with **data, not rhetoric**: CAPM's delta is precisely the strict A1–A6 semantics none of the competitors enforce together.

10.4 The intellectual arc (how the finished paper reads)

1. **Problem on a real system** — laundering demonstrated on a real multi-runtime deployment.
2. **Mechanism (CAPM)** — origin-bound, monotone, externally-verified manifest chain; monotonicity introduced as *design rationale.*
3. **Decomposition** — under the explicit adversary model, transitive laundering reduces to the single origin-integrity residual.
4. **WGOT** — the residual's budgeted targeting (attacker) and its dual (defender hardening) over one residual-risk score.
5. **System & deployment** — one coherent system with measured cost and a tunable security/utility operating point.
6. **Honest limits** — what CAPM does *not* do; the residual is *managed, not eliminated.*

Part 11 — Glossary

- **Agent** — an LLM that can act (call tools, fetch data, message other agents).
- **Principal** — the human/agent at the top who acts on the final answer.
- **Hop** — one agent-to-agent (or agent-to-source) step in the chain.
- **Claim** — an atomic unit of content whose trust we track.
- **Origin** — the first acquisition point of a claim (the web page / API / DB).
- **Acquisition wrapper** — the trusted component that signs an *origin observation* at the moment content enters the system; the **TCB** for origin provenance.
- **Origin observation** — the signed tuple `⟨content_hash, source, timestamp, retriever_identity, source_class_evidence, acquisition_context, nonce⟩`.
- **Manifest** — the hash-linked chain of per-hop signed records.
- **Warrant** — the graded trust value the *verifier* assigns from the manifest (never from the text).
- **Verifier** — the component *outside the LLM* that checks signatures and computes warrant at the principal-facing boundary.
- **Source class** — the origin's trust ceiling (STRONG → NONE), set by the deterministic evidence policy.

- **Monotone (non-increasing)** — the rule that any hop can only keep or lower warrant, never raise it: `T(w) ≤ w`.
- **Min-bounding** — composing multiple inputs takes the **minimum** warrant.
- **A3 safe rule** — if a transform's fidelity isn't verifiable, warrant must drop (unverifiable = treated as generation).
- **Capture / compromise** — the attacker binds high warrant to low-integrity content, usually by owning the source or wrapper.
- **Residual** — what's left after the defense works: genuine origin/wrapper compromise (A7). *Managed, not eliminated.*
- **WGOT** — Warrant-Gradient Origin Targeting; the budgeted optimization over the residual-risk score `w·r·p`, serving attacker (maximize) and defender (minimize).
- **TCB (Trusted Computing Base)** — the minimal set of components that must be correct for the security guarantee to hold (here: wrapper + keys + verifier).
- **ASR (Attack Success Rate)** — the fraction of attacks that achieve laundering success (`W_out > W_origin*`); the headline security metric.
- **SAGA** — the NDSS'26 agent-identity/secure-channel architecture used here as the identity substrate.
- **PROV / PROV-AGENT** — W3C provenance-recording standards (ungraded).
- **CaMeL** — capability-tagging defense that works within a single runtime.
- **C2PA** — signed-manifest content-credential standard (the container pattern).
- **DID / VC** — Decentralized Identifiers / Verifiable Credentials (W3C identity).

Part 12 — A worked end-to-end example

Let's run one claim through the whole machine to make it concrete.

Scenario. Your agent asks for a drug-dosage fact. The chain is:

```
None
  Editable wiki page  ⟶  Retriever C  ⟶  Summarizer B  ⟶  Your agent A  ⟶
YOU
```

Step 1 — acquisition. The retriever's **wrapper** fetches the wiki page. It observes: valid TLS, **no content signature**, host is a **known editable platform**. By the deterministic policy (Part 6) it assigns **WEAK / editable-source**, and because the page has a visible author/revision id, it binds to the **sub-origin** (that author), not the wiki domain. It signs the origin observation:

```
None
  R0 = ⟨ hash(page_bytes), wiki_URL#rev12345, t, retrieverC_id,
         {tls:valid, content_sig:none, editable:true, author:user99},
         "http_fetch", nonce ⟩        − signed by retrieverC
  ⇒  w₀ = ceiling(WEAK)
```

Step 2 — relay + summarize. Summarizer B condenses the text — a **semantic** transform whose fidelity is **not** verifiable. By the **A3 safe rule**, T_1 is the maximum penalty:

```
None
  w₁ = T₁(w₀) = ⊥ (generation floor)          and  w₁ ≤ w₀  ✓ (monotone)
```

B signs R1 (embedding `hash(R0)`).

Step 3 — your agent composes. Suppose A also pulled a STRONG fact from a signed API. Composition is **min-bounded**:

```
None
  w_comp = min( w_from_wiki , w_from_API ) = min( ⊥ , STRONG ) = ⊥
```

The STRONG source **cannot rescue** the weak one. (This is A6 stopping aggregation.)

Step 4 — verification, outside the LLM. The verifier:

- checks every signature back to the registry → all valid ✓
- checks the hash chain R0→R1→R2 is intact → no omission ✓
- computes final warrant `w_comp = ⊥`
- compares to the floor → **below floor**

Verdict: quarantine (or heavy down-weight). You are *not* handed an authoritative-looking dosage. Laundering is **prevented**: even though three reputable agents relayed a beautifully written paragraph, the warrant never rose above the WEAK origin's ceiling — exactly `W_out ≤ W_origin*`, the negation of laundering success.

Where would it *still* leak? Only if the attacker had **compromised the wrapper or the source itself** — forged the channel evidence so the wiki page was observed as a STRONG signed-API origin. That is the **residual** (Part 8): genuine origin-integrity compromise. And even then, the manifest still **names** the compromised wrapper, enabling **attribution + quarantine** afterward (C5).

Closing summary in one paragraph

LLM agents now relay information across organizations, and a lie planted at a weak origin can arrive looking authoritative simply because trusted agents passed it along — **information laundering**, a confusion of *trust in the deliverer* with *trust in the information*. Prior work secures *who speaks* (identity/SAGA), *records* lineage (PROV), controls flow *within one runtime* (CaMeL), checks citation *presence*, or reads *text* (semantic filters) — but **none** binds a **graded** trust value to the **true origin** and keeps it **provably non-increasing** across cross-organizational hops. **CAPM** does: an acquisition wrapper signs an evidence-classified **origin observation**; a hash-linked, per-hop-signed **manifest** carries it; and an **external verifier** computes warrant as a **monotone, min-bounded** function that **only ever falls**, with the **A3 safe rule** keeping any unverifiable transform on the safe side — so security never depends on fooling an ML model. Under the explicit A1–A6 adversary model, a **Residual Reduction Theorem** proves every transit attack is blocked, collapsing the entire threat to **one** residual: **origin/wrapper compromise** — which the **WGOT** optimization then turns into a single residual-risk score that simultaneously reveals the attacker's best target and the defender's best hardening order. The contribution is **the deployable system**; the work is scrupulously honest about what is built (real crypto and warrant logic), what is simulated (real transport and real acquisition), and what remains (Builds A/B/C) before the full systems claim can be made.